

DW_ram_2r_w_s_lat

Sync. Write-Port, Async. Dual Read-Port RAM (Latch-Based)

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

Features and Benefits

- Parameterized word depth
- Parameterized data width
- Synchronous static memory

Revision History

Description

DW_ram_2r_w_s_lat implements a parameterized, synchronous, three-port static RAM consisting of one write port and two read ports. The RAM can perform simultaneous read and write operations.

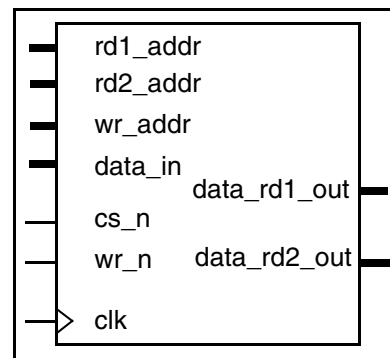


Table 1-1 Pin Description

Pin Name	Width	Direction	Function
clk	1 bit	Input	Clock
cs_n	1 bit	Input	Chip select, active low
wr_n	1 bit	Input	Write enable, active low
rd1_addr	$\text{ceil}(\log_2[\text{depth}])$ bit	Input	Read1 address bus
rd2_addr	$\text{ceil}(\log_2[\text{depth}])$ bits	Input	Read2 address bus
wr_addr	$\text{ceil}(\log_2[\text{depth}])$ bits	Input	Write address bus
data_in	<i>data_width</i> bits	Input	Input data bus
data_rd1_out	<i>data_width</i> bits	Output	Output data bus for read1
data_rd2_out	<i>data_width</i> bits	Output	Output data bus for read2

Table 1-2 Parameter Description

Parameter	Values	Description
data_width	1 to 256 Default: None	Width of <i>data_in</i> and <i>data_out</i> buses
depth	2 to 256 Default: None	Number of words in the memory array (address width)

Table 1-3 Synthesis Implementations

Implementation Name	Function	License Feature Required
str	Synthesis model	DesignWare

Table 1-4 Simulation Models

Model	Function
DW06.DW_RAM_2R_W_S_LAT_CFG_SIM	VHDL simulation configuration
dw/dw06/src/DW_ram_2r_w_s_lat_sim.vhd	VHDL simulation model source code
dw/sim_ver/DW_ram_2r_w_s_lat.v	Verilog simulation model source code

Write data enters the RAM through the `data_in` input port, and is read through either the `data_rd1_out` port or the `data_rd2_out` port. The RAM is constantly reading regardless of the state of `cs_n`.

The `rd1_addr`, `rd2_addr`, and `wr_addr` ports are used to address the *depth* words in the memory. If `rd1_addr` or `rd2_addr` contains a value beyond the maximum *depth* value, that corresponding output port is driven low. For example, if `rd1_addr` = 7 and *depth* = 6, then the `data_rd1_out` bus is driven low. If `rd2_addr` is beyond the maximum *depth*, then `data_rd2_out` is driven low.

For `wr_addr` beyond the maximum *depth* (for example, `wr_addr` = 7 and *depth* = 6), nothing occurs and the data is lost. No warnings are given during simulations when an address beyond the scope of *depth* is used.



Attention

This component contains enable signals for internal latches that are derived from the `wr_n` port. To keep hold times to a minimum, you should consider instances of this component to be individual floorplanning elements.

Chip Selection, Reading and Writing

The `cs_n` input is the chip select, active low signal that enables data to be written to the RAM. The RAM is constantly reading, regardless of the state of `cs_n`.

When `cs_n`, `wr_n` (write enable, active low), and `clk` are low, `data_in` is transparent to the internal memory cell being accessed by `wr_addr` (`data_in` = output data of the memory cell). Therefore, during the period when `clk` is low, a change in `data_in` is reflected on the output of the internal memory cell being accessed. If one of the read address buses is the same value as `wr_addr` and `wr_n` is low, `data_in` equals the output data when `clk` is low. Data is captured into the memory cell on the rising edge of `clk`.

When `cs_n` is high, writing to the RAM is disabled.

Application Notes

DW_ram_2r_w_s_lat is intended to be used as small scratch-pad memory, programmable lookup tables, and writable control storage. Because DW_ram_2r_w_s_lat is built from the cells within the ASIC cell library, it should be kept small to obtain an efficient implementation. If a larger memory is required, you should consider using a hard macro RAM from the ASIC library in use.

Suppressing Warning Messages During Verilog Simulation

The Verilog simulation model includes macros that allow you to suppress warning messages during simulation.

To suppress all warning messages for all DWBB components, define the DW_SUPPRESS_WARN macro in either of the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_SUPPRESS_WARN
```

- Or, include a command line option to the simulator, such as:

```
+define+DW_SUPPRESS_WARN (which is used for the Synopsys VCS simulator)
```

The warning messages for this model include the following:

- If values other than 1 or 0 are present on a clock port, the following message is displayed:

```
WARNING: <instance_path>.<clock_name>_monitor:  
at time = <timestamp>, Detected unknown value, x, on <clock_name> input.
```

To suppress only this warning message for all DWBB components, use the following macro:

- Define the DW_DISABLE_CLK_MONITOR macro. You can define this macro in the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_DISABLE_CLK_MONITOR
```

- Or, include a command line option to the simulator, such as:

```
+define+DW_DISABLE_CLK_MONITOR (which is used for the Synopsys VCS simulator)
```

This message is also suppressed using the DW_SUPPRESS_WARN macro explained earlier.

Related Topics

- [Memory – Synchronous RAMs Listing](#)
- [DesignWare Building Block IP User Guide](#)

HDL Usage Through Component Instantiation - VHDL

```

library IEEE,DWARE;
use IEEE.std_logic_1164.all;
use DWARE.DWpackages.all;
use DWARE.DW_foundation_comp.all;

entity DW_ram_2r_w_s_lat_inst is
  generic (inst_data_width : INTEGER := 8;   inst_depth : INTEGER := 8 );
  port (inst_clk      : in std_logic;
        inst_cs_n     : in std_logic;
        inst_wr_n     : in std_logic;
        inst_rd1_addr : in std_logic_vector(bit_width(inst_depth)-1 downto 0);
        inst_rd2_addr : in std_logic_vector(bit_width(inst_depth)-1 downto 0);
        inst_wr_addr  : in std_logic_vector(bit_width(inst_depth)-1 downto 0);
        inst_data_in  : in std_logic_vector(inst_data_width-1 downto 0);
        data_rd1_out_inst : out std_logic_vector(inst_data_width-1 downto 0);
        data_rd2_out_inst : out std_logic_vector(inst_data_width-1 downto 0)
  );
end DW_ram_2r_w_s_lat_inst;

architecture inst of DW_ram_2r_w_s_lat_inst is
begin

  -- Instance of DW_ram_2r_w_s_lat
  U1 : DW_ram_2r_w_s_lat
    generic map ( data_width => inst_data_width,  depth => inst_depth )
    port map (clk => inst_clk,  cs_n => inst_cs_n,  wr_n => inst_wr_n,
              rd1_addr => inst_rd1_addr,  rd2_addr => inst_rd2_addr,
              wr_addr => inst_wr_addr,  data_in => inst_data_in,
              data_rd1_out => data_rd1_out_inst,
              data_rd2_out => data_rd2_out_inst );

end inst;

-- pragma translate_off
configuration DW_ram_2r_w_s_lat_inst_cfg_inst of DW_ram_2r_w_s_lat_inst is
  for inst
  end for; -- inst
end DW_ram_2r_w_s_lat_inst_cfg_inst;
-- pragma translate_on

```

HDL Usage Through Component Instantiation - Verilog

```

module DW_ram_2r_w_s_lat_inst(inst_clk, inst_cs_n, inst_wr_n, inst_rd1_addr,
                             inst_rd2_addr, inst_wr_addr, inst_data_in,
                             data_rd1_out_inst, data_rd2_out_inst );

    parameter data_width = 8;
    parameter depth = 8;
    `define bit_width_depth 3 // ceil(log2(depth))

    input inst_clk;
    input inst_cs_n;
    input inst_wr_n;
    input [`bit_width_depth-1 : 0] inst_rd1_addr;
    input [`bit_width_depth-1 : 0] inst_rd2_addr;
    input [`bit_width_depth-1 : 0] inst_wr_addr;
    input [data_width-1 : 0] inst_data_in;
    output [data_width-1 : 0] data_rd1_out_inst;
    output [data_width-1 : 0] data_rd2_out_inst;

    // Instance of DW_ram_2r_w_s_lat
    DW_ram_2r_w_s_lat #(data_width, depth)
        U1 (.clk(inst_clk), .cs_n(inst_cs_n), .wr_n(inst_wr_n),
           .rd1_addr(inst_rd1_addr), .rd2_addr(inst_rd2_addr),
           .wr_addr(inst_wr_addr), .data_in(inst_data_in),
           .data_rd1_out(data_rd1_out_inst),
           .data_rd2_out(data_rd2_out_inst) );
endmodule

```

Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet with visible change bars, click [here](#).

Date	Release	Updates
September 2024	DWBB_202409.0	■ Updated date and version; no technical changes
December 2023	DWBB_202312.0	■ Updated date and version; no technical changes
July 2020	DWBB_201912.5	■ Adjusted content and title of “Suppressing Warning Messages During Verilog Simulation” on page 3 and added the DW_SUPPRESS_WARN macro
October 2019	DWBB_201903.5	■ Added the “Disabling Clock Monitor Messages” section
January 2019	DWBB_201806.5	■ Updated example in “HDL Usage Through Component Instantiation - VHDL” on page 5 ■ Added this Revision History table and the document links on this page

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.

www.synopsys.com